

DEEP LEARNING

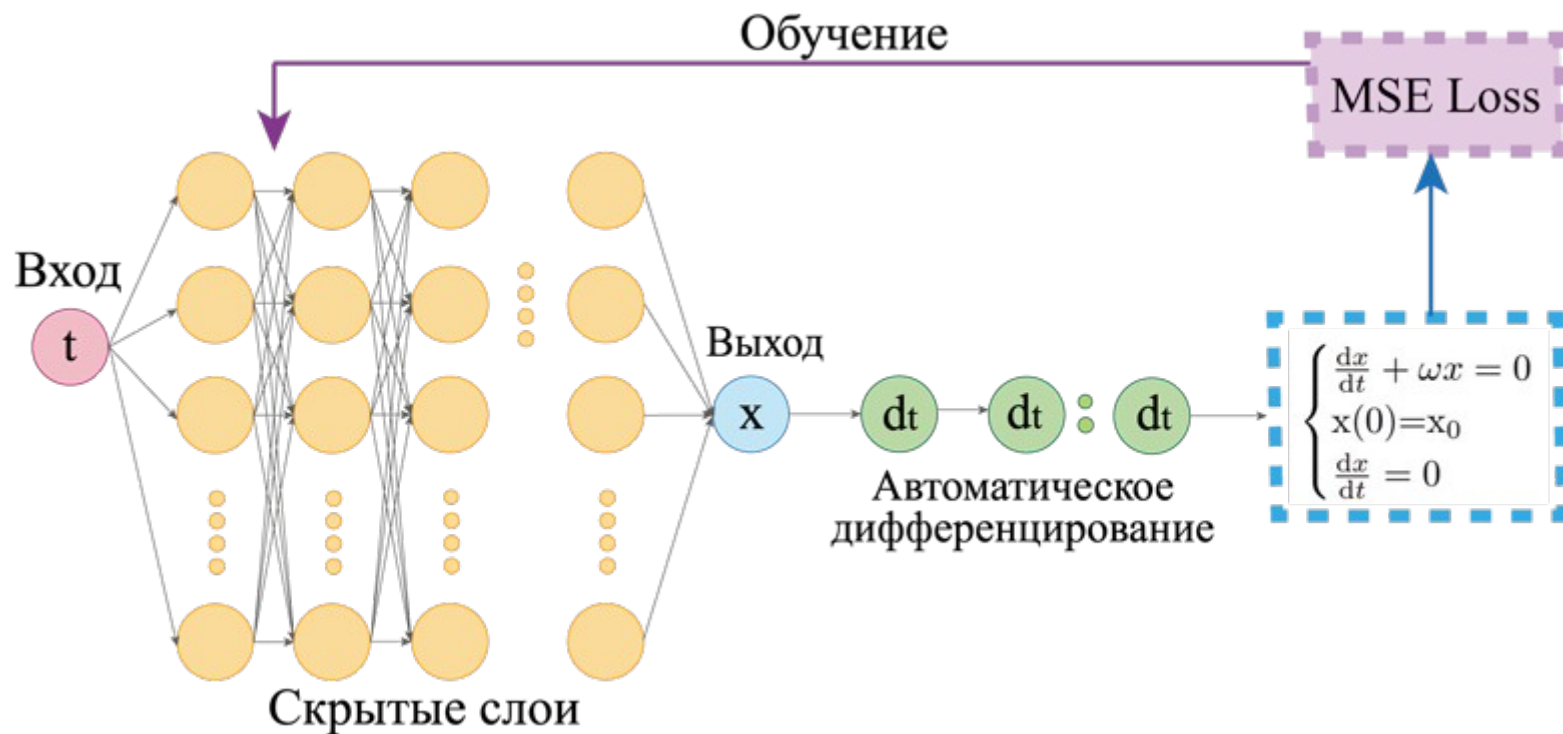
Сеть прямого распространения

ГУУ, 3-й курс 2026, 2-й семестр

ПЛАН

- Проблема "черного ящика"
- Механика автоградиента
- Архитектура PINN на примере ОДУ
- Пример с PDE
- Ограничения

PINN

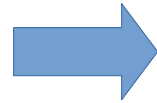


Проблема черного ящика

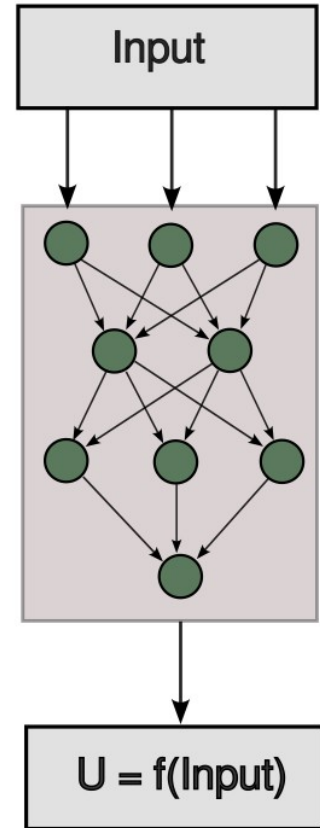
- Нейросеть хорошо работает для решения задач интерполяции
- Экстраполяция — некорректно поставленная задача

Решение задачи в лоб

$$\frac{dU}{dt} = f(U, t)$$



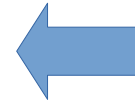
Сбор данных U_i, t_i



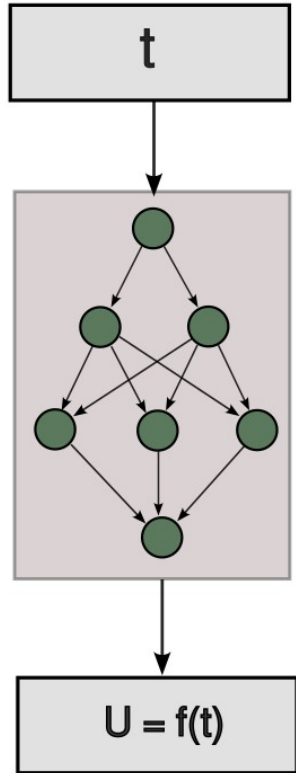
$$E = (U - \hat{U}(w))^2$$



$$\|U - \hat{U}(w)\|_{L_2} \rightarrow \min$$



Учитываем физику процесса



$$\frac{du}{dt} = u \sin t$$

- Ищем решение как функцию параметров нейросети $u = u(w)$
- Используем автоградиент для нахождения du/dt

$$E = a_0 (u - \hat{u}(w))^2 + a_1 \left(\hat{u}(w) \sin(t) - \frac{d\hat{u}(w)}{dt} \right)^2 + a_2 (\hat{u}(w, t_0) - u(t_0))^2$$

Механика автоградиента

```
import torch
```

```
# Создаем входной тензор (время t), разрешаем вычисление градиента по нему  
t = torch.tensor([[1.0], [2.0]], requires_grad=True)
```

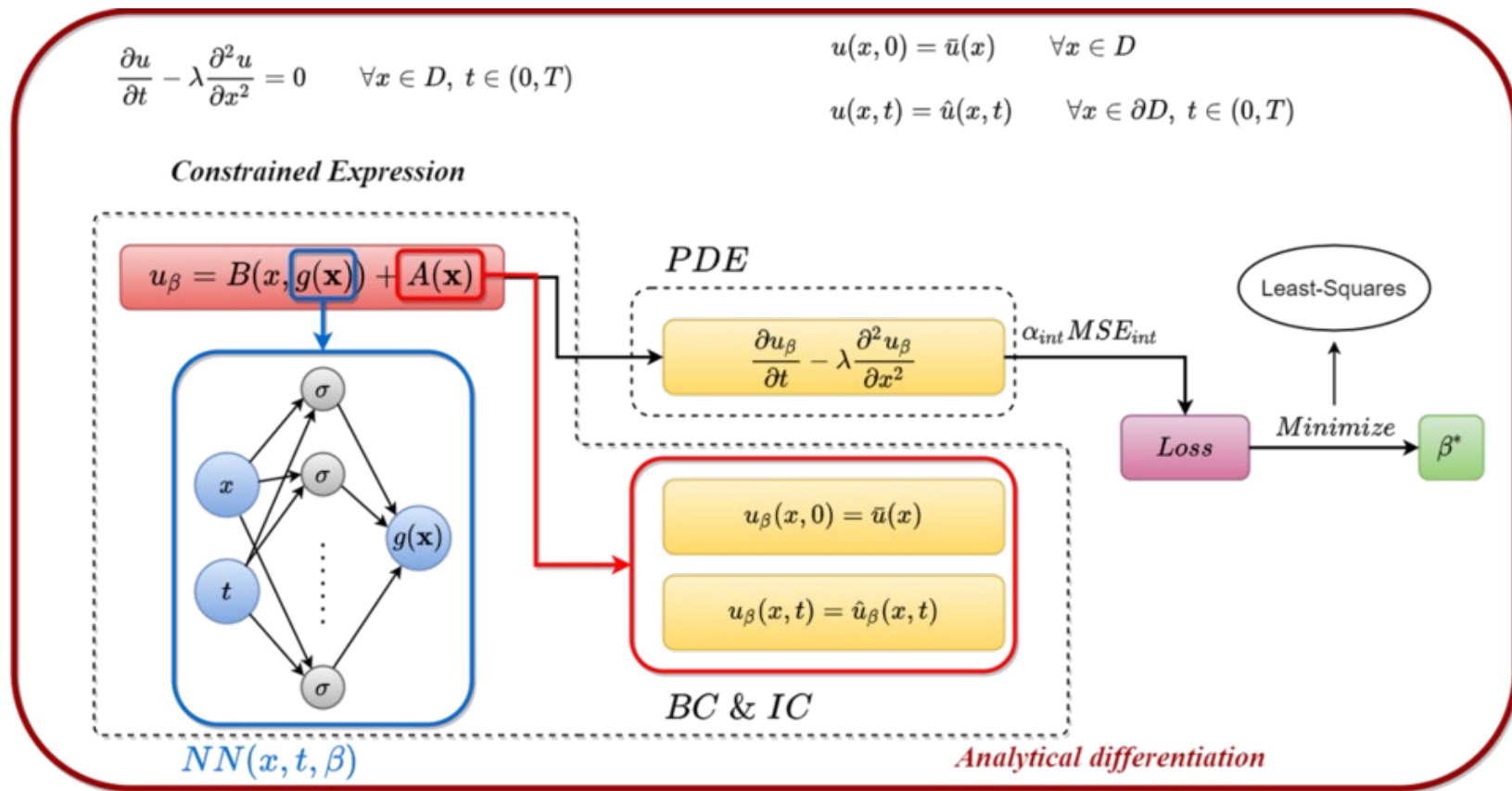
```
# Простая "нейросеть" (один слой для примера)  
model = torch.nn.Sequential(torch.nn.Linear(1, 10), torch.nn.Tanh(), torch.nn.Linear(10, 1))
```

```
# Прямой проход  
y = model(t)
```

```
# Вычисляем производную dy/dt (физическую скорость)  
# grad_outputs - это веса для производной, обычно тензор из единиц  
dy_dt = torch.autograd.grad(y, t, grad_outputs=torch.ones_like(y), create_graph=True)[0]
```

```
print(f"Производная в точках t: {dy_dt}")
```

От ОДУ к уравнениям в частных производных



Подводные камни

- Функции активации: ReLU нельзя использовать, так как его вторая производная везде ноль. Нужны гладкие активации (Tanh, SiLU/Swish)
- Баланс весов: Физическая ошибка и ошибка данных могут «перетягивать канат», из-за чего сеть плохо учится.
- Сходимость: PINN могут застревать в локальных минимумах там, где классические численные методы (Runge-Kutta) работают стабильно.